

Hoy la app no guarda absolutamente nada en el dispositivo

Diagnóstico de la arquitectura de datos actual y plan para convertirla en offline-first sin reescribir el proyecto. El hallazgo que ordena todo lo demás: no existe ninguna base de datos local, el login es obligatorio, y cada acción del juego escribe directo a Firestore o se pierde.

Rama develop · 7ad2ce2 **Stack** Flutter · Riverpod 3 · Firebase Auth · Cloud Firestore

Alcance Diagnóstico + plan. Sin código. **Fecha** 31 ago 2026

Diagnóstico

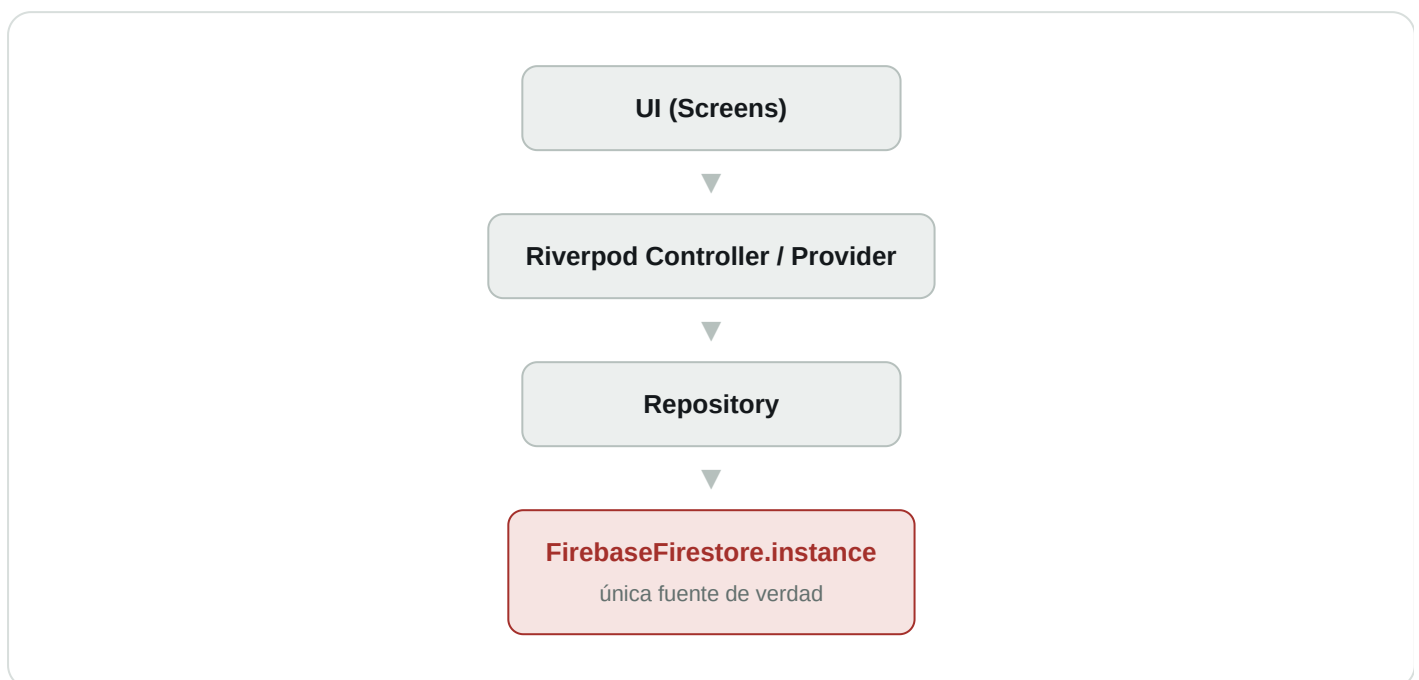
01 Arquitectura actual

Feature-first bien organizada, con una capa de datos que va directa a la nube. No es offline-first: es online-only con un caché que el usuario no controla.

La estructura por features

`(lib/features/<feature>/{controller, repository, model, widget})`

es correcta y no hay que tocarla. El problema está en la dirección del flujo de datos:



El patrón Repository **ya existe y está bien aplicado**: la UI no toca Firestore directamente en ningún sitio. Eso es exactamente el punto de entrada que necesitamos, porque insertar una capa local significa cambiar la implementación de tres repositories, no reescribir pantallas.

Lo que falta es la mitad izquierda del diagrama: no hay *local data source*, no hay cola de sincronización y no hay concepto de usuario sin cuenta.

02 Sistema de almacenamiento local actual

HALLAZGO PRINCIPAL

No existe. Cero. Busqué en todo `lib/`: no hay `SharedPreferences`, ni `Hive`, ni `Isar`, ni `sqflite`, ni `Drift`, ni acceso a `path_provider`.
`shared_preferences` y `path_provider` aparecen en `pubspec.lock` únicamente como dependencias transitivas de `google_sign_in` y `Firebase`.

La única persistencia en el dispositivo hoy es la **caché offline de Cloud Firestore**, que el SDK de móvil activa por defecto. Conviene entender exactamente qué resuelve y qué no, porque es fácil confundirla con una solución `offline-first`:

ESCENARIO	CACHÉ DE FIRESTORE
Usuario autenticado, pierde red a mitad de sesión	Funciona — lee de caché y encola escrituras
Usuario autenticado, reinicia la app sin red	Parcial — la caché sobrevive, pero <code>authStateChanges</code> y los <code>snapshots()</code> tardan y la app arranca en un estado indeterminado
Usuario sin cuenta	Nada. No hay UID, no hay documento, no hay caché
Cola de escrituras entre reinicios	Persiste, pero es opaca: no puedes inspeccionarla, priorizarla, reintentar con <code>backoff</code> ni saber qué falló
Datos que no son de Firestore (vidas, preferencias, progreso de reto)	Nada

Es decir: hay caché, pero está atada a un UID y es una caja negra. No sirve como *source of truth* ni como cola de sincronización auditable, y no cubre el caso central del producto — jugar sin cuenta.

03 Sistema Firebase actual

Tres paquetes: `firebase_core`, `firebase_auth`, `cloud_firestore`. Sin Cloud Functions, sin Storage, sin Remote Config, sin Analytics.

El arranque bloquea:

```
// lib/main.dart
await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
await FlutterLocalization.instance.ensureInitialized();
runApp(const ProviderScope(child: MyApp()));
```

Nada se dibuja hasta que Firebase termina de inicializar. Es rápido en el caso feliz, pero es un bloqueo estructural que contradice directamente los puntos 35 y 36 de tu brief.

COLECCIÓN	DECLARADA EN RULES	USADA EN CÓDIGO	ESCRITA POR
users/{uid}	sí	sí	UserReposito
users/{uid}/matches/{matchId}	sí	sí	MatchReposito
users/{uid}/levels/{levelId}	no	sí	LevelReposito
users/{uid}/achievements/{id}	sí	no	nadie
daily_challenges/{id}	sí	no	nadie
leaderboards/{id}	sí	no	nadie
user_index/{uid}	sí	no	nadie

Dos desalineaciones que importan: `levels` se escribe sin regla que la cubra — hoy pasa por el `match /users/{uid}` padre solo porque las reglas de subcolección no se heredan salvo por comodín, así que conviene revisarlo explícitamente. Y hay cuatro colecciones declaradas que nadie usa: no son deuda, son el esqueleto ya reservado para amigos, rankings y retos. Las reutilizaremos en vez de inventar nuevas.

04 Sistema de autenticación actual

Firebase Auth con tres métodos: email/contraseña, Google Sign-In y teléfono (SMS). La implementación es limpia — `AuthController` expone `AsyncValue` y las pantallas conducen su spinner y su error con `ref.listen`. **No hay que reemplazarla.**

El problema es la puerta, no el mecanismo:

```
// lib/features/auth/splash/splash_page.dart
final isSignedIn = ref.read(firebaseAuthProvider).currentUser != null;
Navigator.pushReplacementNamed(
  context,
  isSignedIn ? RoutePaths.home : RoutePaths.login, // ← no hay tercera vía
);
```

Un usuario nuevo aterriza obligatoriamente en el login. No existe «Ahora no». Y aunque existiera, no le serviría de nada: todos los controladores de gameplay lanzan al no haber sesión.

```
// se repite en GameController, UserController y LevelController
final user = _auth.currentUser;
if (user == null) throw Exception('No authenticated user');
```

Firebase Auth ofrece `signInAnonymously()`, que a primera vista resuelve esto. **No lo recomiendo como base.** Requiere red en el primer arranque (justo el momento que queremos blindar), crea un UID desechable que se pierde si el usuario borra datos, y ata la identidad local a un servicio que precisamente queremos que sea opcional. El perfil local debe ser nuestro, con un UUID generado en el dispositivo.

05 Estado actual de la Home

Tras el rediseño de la fase anterior, la Home tiene la composición correcta y lee de tres providers:

DATO	PROVIDER	ORIGEN REAL	¿OFFLINE?
Nombre, nivel, XP, racha	homeSummaryProvider	currentUserProvider → Firestore snapshots()	no sin cuenta
Monedas	walletControllerProvider	igual, vía currentUserProvider	no sin cuenta
Última partida	homeControllerProvider	userMatchHistoryProvider → Firestore	no sin cuenta
Reto diario	—	constante en la pantalla	estático

La buena noticia arquitectónica: **la Home ya observa providers, no Firestore.**

`homeSummaryProvider` es un `Provider` derivado y `homeControllerProvider` un `AsyncNotifier`. Cambiar su fuente de Firestore a la base local es un cambio contenido dentro de esos tres providers — cero cambios en widgets. La estructura del punto 18 de tu brief ya está a mitad de camino.

La mala: sin sesión, `currentUserProvider` emite `null` y `userMatchHistoryProvider` emite lista vacía. La Home se renderiza — pero con nivel 1, 0 XP, 0 monedas y sin partidas, para siempre.

06 Repositories existentes

REPOSITORY	RESPONSABILIDAD	VEREDICTO
UserRepository	CRUD de users/{uid}, incluidos addCoins, addXp, recordGameWin con FieldValue.increment	Reutilizar como <i>remote data source</i> . Los increments ya son la base correcta para sincronizar.
MatchRepository	Escribe y lee users/{uid}/matches	Reutilizar la escritura. getUserStats () hay que retirarlo (ver §13).
LevelRepository	Inicializa 50 niveles y marca completados	Repensar. 50 escrituras en el registro para datos deterministas, y un bug de tipo en completeLevel.

Los tres reciben `Firestore`? por constructor con ??

`Firestore.instance` — ya son inyectables, lo que facilita los tests y el sustituir la instancia. Es el mismo constructor que usaremos para inyectar el `data source` local.

07 Providers existentes

PROVIDER	TIPO	ESTADO
firebaseAuthProvider · googleSignInProvider · authStateChangesProvider	Provider / Stream	Reutilizar
currentUserProvider	StreamProvider<AppUser? >	Redirigir a la base local
userMatchHistoryProvider	StreamProvider	Redirigir a la base local
userStatsProvider	StreamProvider	Retirar — descarga toda la colección
userLevelsProvider · currentLevelProvider	Stream / Future	Redirigir a la base local
walletControllerProvider	AsyncNotifier<int>	Reescribir (ver §08)
homeSummaryProvider · homeControllerProvider	Provider / AsyncNotifier	Solo cambia la fuente
livesControllerProvider	Notifier	Sin persistencia; se reinicia a 5 vidas en cada arranque
connectionStatusControllerProvider	Notifier	Reutilizar, con un matiz (ver §10)
shopControllerProvider · friendsControllerProvider · versusControllerProvider · levelMapControllerProvider · profileControllerProvider	AsyncNotifier	Mocks con Future.delayed y datos escritos a mano

`levelMapControllerProvider` merece mención aparte: devuelve cuatro nodos fijos y **no tiene ninguna relación** con `LevelRepository` ni con `userLevelsProvider`. Hay dos sistemas de niveles paralelos en el proyecto, y el que ve el jugador al pulsar «Jugar Solo» es el falso.

08 Estado actual del Wallet

EL WALLET NO GUARDA NADA, EN NINGÚN SITIO

`WalletController.add()` y `spend()` mutan únicamente el estado de Riverpod. No llaman a `UserRepository`, no escriben a Firestore, no persisten localmente. Y como `build()` deriva de `currentUserProvider`, cualquier `snapshot` entrante o cualquier invalidación revierte el saldo al valor de la nube.

El efecto en el juego es concreto: el jugador gasta 200 monedas en la tienda, ve el saldo bajar, cambia de pantalla y las monedas vuelven. Nunca se descuentan de verdad.

Peor todavía, el mismo evento de fin de partida produce **tres recompensas distintas calculadas en tres sitios distintos**:

DÓNDE	QUÉ CALCULA	DÓNDE ACABA
<code>BoardController._calculateRewards</code>	monedas según score, tiempo y movimientos	overlay de victoria — lo que el jugador ve
<code>BoardPage ref.listen</code>	<code>_victoryCoinsReward = 50</code> fijo	saldo en memoria — se pierde
<code>Match.calculateRewards</code>	otra fórmula distinta	Firestore — lo que el jugador <i>tiene</i>

Y el `ref.listen` de `BoardPage` dispara con `isCompleted`, que también se activa cuando **se acaba el tiempo**. Perder una partida por tiempo abona 50 monedas en pantalla.

09 Estado actual de Match

El modelo `Match` está bien pensado: `ID` propio, `gameMode`, `score`, movimientos, tiempo, recompensas, `won`, `playedAt` y campos de oponente ya previstos para `versus`. Se guarda en `users/{uid}/matches/{matchId}` con un `Uuid().v4()` generado en el cliente — que es exactamente lo que necesitamos para `offline`.

Tres problemas:

- **No es idempotente.** El UUID se genera *dentro* de `completeSoloGame()`. Si esa función se reintentra tras un fallo de red, genera un ID nuevo: partida duplicada y doble `increment` de monedas y XP. El ID debe nacer con la partida, no con el intento de escritura.
- **`playedAt` es reloj del cliente**, sin `serverTimestamp` de contraste. Necesario para `offline`, pero hay que registrar también la hora del servidor al sincronizar para poder detectar manipulación.
- **Se guarda antes de saber si hay red**, y el `try/catch` de `BoardController` se traga el error con un `print`. La partida se pierde en silencio.

10 Connectivity existente

`ConnectionStatusController` escucha `connectivity_plus` y expone un enum `online | offline`. Lo consume `ConnectivityBanner`, envuelto alrededor de toda la app en `MaterialApp.builder`. La base es correcta y se reutiliza.

Dos matices para el Sync Engine:

- `connectivity_plus` reporta la **interfaz**, no la alcanzabilidad. Wi-Fi de hotel con portal cautivo devuelve `online`. Sirve como disparador barato para intentar sincronizar; no sirve como garantía. La verdad final la da el resultado de la escritura a Firestore.
- `build()` devuelve `ConnectionStatus.online` optimista mientras resuelve el `checkConnectivity()` asíncrono. Para un banner da igual; para decidir si se intenta sincronizar, conviene un tercer estado `unknown`.

11 Firestore rules

```
match /users/{uid} {  
  allow read, write: if request.auth.uid == uid;  
  allow read: if true;           // ← el documento entero, públicamente  
}
```

DOS PROBLEMAS GRAVES, UNO DE ELLOS DE PRIVACIDAD

1. Fuga de datos personales. `allow read: if true` sobre `users/{uid}` expone el documento completo a cualquiera, sin autenticar — incluidos `email` y `phoneNumber`. El comentario dice «para amigos/leaderboards», pero eso solo necesita nombre, avatar y nivel. Lo mismo aplica a `matches`. En una app dirigida a niños, esto no es un detalle de seguridad: es un problema de cumplimiento.

2. Cero validación de escritura. Cualquier usuario autenticado puede hacer `set({totalCoins: 99999999, level: 999})` sobre su propio documento. No hay `allowlist` de campos, ni monotonía, ni topes por escritura.

Añade a eso que `request.auth.uid` se evalúa sin comprobar `request.auth != null`: en las reglas de Firestore eso no falla, pero es frágil y conviene escribirlo explícito.

12 Firestore indexes

Dos índices compuestos sobre `matches: won + playedAt DESC` y `gameMode + playedAt DESC`. Corresponden exactamente a `getUserWins()` y `getUserMatchesByMode()`. Están bien definidos y `fieldOverrides` está vacío.

Observaciones menores: ninguno de los dos métodos se llama hoy desde la UI, y en un modelo offline-first la mayoría de estas consultas pasarán a resolverse contra la base local — donde son consultas SQL triviales. Los índices se quedan (los necesitaremos para rankings y para la reconstrucción de historial en un dispositivo nuevo), pero no habrá que ampliarlos en esta fase.

13 Problemas de arquitectura

Ordenados por impacto. Los cinco primeros son bloqueantes para offline-first; el resto son defectos que conviene arreglar mientras tocamos esa capa.

01 No hay fuente de verdad local

Todo el gameplay depende de una escritura remota inmediata. Es la causa raíz de la que cuelgan casi todos los demás puntos.

02 Jugar exige cuenta

Splash enruta a login sin tercera vía, y los tres controladores de gameplay lanzan `Exception('No authenticated user')`. No existe el concepto de jugador local.

`splash_page.dart · game_controller.dart · user_controller.dart · level_controller.dart`

03 El wallet nunca persiste

Gastar y ganar monedas solo muta estado de Riverpod, que se revierte al primer snapshot. Ver §08.

`wallet_controller.dart`

04 El nivel nunca sube

`addXp` incrementa `currentXp` en Firestore, pero nada escribe jamás `level`. `GameController._updateLevelProgression()` es un cuerpo vacío cuyo comentario afirma que «la lógica ya está en el user controller» — no lo está. El jugador se queda en Nivel 1 con XP creciendo sin límite, y la barra de la Home nunca se completa.

`game_controller.dart:117 · user_controller.dart`

05 La racha no existe

`bestStreak` está en el modelo, se lee en la Home y en el Perfil, y `UserRepository.updateBestStreak()` está implementado — pero **nadie lo llama nunca**. La racha es siempre 0.

`user_repository.dart · sin llamadas`

06

Tres cálculos de recompensa divergentes

El jugador ve un número, el saldo suma otro y Firestore guarda un tercero. Y perder por tiempo abona 50 monedas. Ver §08.

07

bestScore se escribe como array sobre un campo int

`LevelRepository.completeLevel` hace `'bestScore'`: `FieldValue.arrayUnion([score])`, pero `GameLevel.fromFirestore` lee `data['bestScore'] as int?`. Tras la primera partida el campo pasa a ser un array y el `cast` devuelve `null` → 0. Además usa `batch.update`, que lanza si el documento no existe.

`level_repository.dart` · `level.dart`

08

getUserStats() descarga la colección entera

Lee *todos* los documentos de `matches` para calcular agregados en el cliente, y `userStatsProvider` lo envuelve en un stream. Con 500 partidas son 500 lecturas facturadas cada vez que se reconstruye. Contradice tu punto 16.

`match_repository.dart`

09

50 escrituras en el registro por usuario nuevo

`initializeLevels()` crea 50 documentos cuyo contenido es 100 % determinista (`generateGameLevels`). Solo `isUnlocked`, `isCompleted` y `bestScore` son datos reales del jugador. La definición debe ser local; solo el progreso merece viajar.

`level_repository.dart`

10

Dos sistemas de niveles paralelos

`LevelMapController` devuelve cuatro nodos escritos a mano y no habla con `LevelRepository`. Es el que ve el jugador desde «Jugar Solo».

`level_map_controller.dart`

11

Las vidas se reinician en cada arranque

`LivesController` es memoria pura con un `Timer` que solo corre con la app abierta. Cerrar y reabrir devuelve 5 vidas: la economía de vidas es evitable sin querer, y el temporizador de recarga miente.

`lives_controller.dart`

12

Errores de red silenciados con `print`

`BoardController._onGameCompleted` y `GameController._updateLevelProgression` capturan y imprimen. Sin cola de reintentos, «silent fail» significa pérdida de progreso.

13

Firebase bloquea el arranque

`await Firebase.initializeApp()` antes de `runApp`. Contradice los puntos 35 y 36.

`main.dart`

14

Un único test, autogenerado

`test/widget_test.dart` es el contador por defecto de Flutter. No hay cobertura de nada.

14 Qué podemos reutilizar

intacto

El patrón Repository

Ninguna pantalla toca Firestore. Insertar la capa local es cambiar implementaciones, no pantallas.

intacto

Firebase Auth completo

Email, Google y teléfono. Solo añadimos una vía que lo esquiva al principio.

intacto

Modelos de dominio

`AppUser`, `Match`, `GameLevel` son POCOs con `toFirestore/fromFirestore`. Solo añadimos `toLocal/fromLocal` y unos campos.

intacto

Increments de Firestore

`recordGameWin` y `addCoins` ya usan `FieldValue.increment`. Es literalmente el mecanismo de resolución de conflictos que necesitamos.

intacto

Connectivity + banner

Se convierte en el disparador del Sync Engine.

intacto

Colecciones reservadas

`user_index`, `daily_challenges`, `leaderboards`, `achievements` ya están en las rules. Cero colecciones nuevas que justificar.

intacto

Providers de la Home

`homeSummaryProvider` y `homeControllerProvider` ya desacoplan la UI. Solo cambia su fuente.

intacto

Design System

No se toca. La sincronización se expresa con `AppBadge` y los tokens que ya existen.

15 Qué debemos modificar

ARCHIVO	CAMBIO
main.dart	Abrir la base local, runApp inmediato, Firebase sin await bloqueante
splash_page.dart	Tercera vía: si hay perfil local → Home. Nunca forzar login.
user_repository.dart	Pasa a ser UserRemoteDataSource; el repository nuevo lo orquesta con el local
match_repository.dart	Igual. Retirar getUserStats(): los agregados salen de la base local
level_repository.dart	Definiciones locales; solo progreso en la nube. Corregir bestScore y el batch.update
wallet_controller.dart	Reescribir sobre el repository local; spend/add generan operaciones
game_controller.dart	Escribir local + encolar. Retirar el stub muerto. Una única fórmula de recompensa.
board_controller.dart · board_page.dart	Eliminar los dos cálculos de recompensa duplicados y el abono de 50 monedas al perder
lives_controller.dart	Persistir vidas y el instante de la última recarga
user.dart (AppUser)	totalXp monótono; level y currentXp pasan a derivados (ver §21)
match.dart	localId, syncStatus, serverPlayedAt
connection_status_controller.dart	Añadir unknown
firestore.rules	Cerrar la fuga y validar escrituras (§22)
level_map_controller.dart	Conectar al sistema de niveles real

16 Qué debemos crear

```
lib/core/database/           // Drift: tablas, DAOs, migraciones
lib/core/database/tables/
lib/core/database/dao/
lib/core/sync/               // SyncEngine, SyncQueue, SyncOperation, políticas de reintent
lib/features/player/         // PlayerProfile local, PlayerRepository, identidad y migració
lib/features/daily_challenge/ // definición cacheada + progreso local
```

Cuatro carpetas. Nada más. Todo lo demás son modificaciones sobre lo que ya existe.

Propuesta

17 Propuesta de Local Data Model

Recomiendo **Drift** (SQLite tipado) como base local. Es la decisión más consecuente de todo el plan, así que la justifico.

REQUISITO DE TU BRIEF	DRIFT	HIVE / SHAREDREFERENCES	ISAR
Relaciones e historial consultable	SQL con joins e índices	clave-valor	consultas propias
Cola de operaciones con estado y reintentos	tabla con índices y transacciones	a mano	sí
Migraciones de esquema versionadas	de primera clase	manuales	limitadas
Streams reactivos para Riverpod	.watch() → StreamProvider	parcial	sí
Tests sin Firebase	NativeDatabase.memory()	requiere temp dir	requiere binarios
Android · iOS · macOS · web	sí (WASM en web)	sí	web parcial
Mantenimiento del paquete	activo	Hive v2 estancado	v4 en beta prolongada

Coste honesto: Drift usa `build_runner`. Son dos dependencias en runtime (`drift`, `sqlite3_flutter_libs`) y dos de desarrollo (`drift_dev`, `build_runner`), más un paso de codegen en el flujo. Lo acepto porque el esquema va a evolucionar durante meses y las migraciones tipadas son exactamente lo que evita corromper el progreso de un jugador que lleva 30 días sin cuenta. Si prefieres cero codegen, **sembast** es la alternativa razonable, a cambio de perder las consultas SQL y las migraciones formales.

`shared_preferences` se queda para lo que es: idioma, sonido, si ya vimos el aviso de cuenta. Nunca para datos de juego.

Esquema

player_profiles

```
localId      TEXT PK      // UUID v4 generado en el 1er arranque. Nunca cambia.
cloudUid     TEXT NULL    // se rellena al crear cuenta
displayName  TEXT
avatarSeed   INT
totalXp      INT          // monótono, de por vida
totalCoins   INT
gamesWon     INT
totalMoves   INT
currentStreak INT
longestStreak INT
lastPlayedDate TEXT      // 'YYYY-MM-DD' en zona local
createdAt    INT
updatedAt    INT
version      INT          // contador para LWW en campos de perfil
```

matches

```
id           TEXT PK      // UUID generado al INICIAR la partida
playerLocalId TEXT FK
gameMode     TEXT
score, moves, secondsElapsed, timeLimit, coinsEarned, xpEarned INT
won          BOOL
playedAt     INT
syncStatus   TEXT        // synced | pending | syncing | failed
// índice: (playerLocalId, playedAt DESC) → «última partida» en 0(1)
```

level_progress // solo progreso. La definición se genera en código.

```
playerLocalId TEXT
levelNumber   INT
isCompleted   BOOL
bestScore     INT          // int, no array
completedAt   INT
PK (playerLocalId, levelNumber)
```

daily_challenge_defs // caché de contenido descargado

```
challengeId TEXT PK, date TEXT, payloadJson TEXT, cachedAt INT
```

daily_challenge_progress

```
playerLocalId TEXT, challengeId TEXT, completed BOOL,
score INT, completedAt INT, syncStatus TEXT
PK (playerLocalId, challengeId)
```

lives_state

```
playerLocalId TEXT PK, current INT, lastRefillAt INT
```

```

sync_operations          // el corazón del motor
  opId      TEXT PK    // UUID – también es el ID del ledger remoto
  type      TEXT       // ADD_XP | EARN_COINS | SPEND_COINS | CREATE_MATCH | ...
  entityType TEXT
  entityId  TEXT
  payloadJson TEXT
  createdAt INT
  status     TEXT      // pending | syncing | synced | failed
  retryCount INT
  nextAttemptAt INT    // backoff exponencial
  lastError  TEXT

```

LA DECISIÓN QUE ELIMINA UNA CLASE ENTERA DE CONFLICTOS

Sustituir **currentXp** (XP dentro del nivel) por **totalXp** (XP acumulado de por vida). Con **totalXp**, **level** y el progreso de la barra pasan a ser *funciones puras* del acumulado y dejan de existir como datos sincronizables. Un contador monótono se resuelve con increments; un nivel almacenado no se resuelve con nada. De paso arregla el bug 04, porque el nivel deja de necesitar que alguien lo escriba.

Migración: para cada usuario existente, $\text{totalXp} = \sum(1000 \times n)$ para $n < \text{level}$, más **currentXp**. Se ejecuta una sola vez, es determinista, y se puede correr en cliente al primer arranque tras la actualización porque solo depende de campos que ya están en el documento.

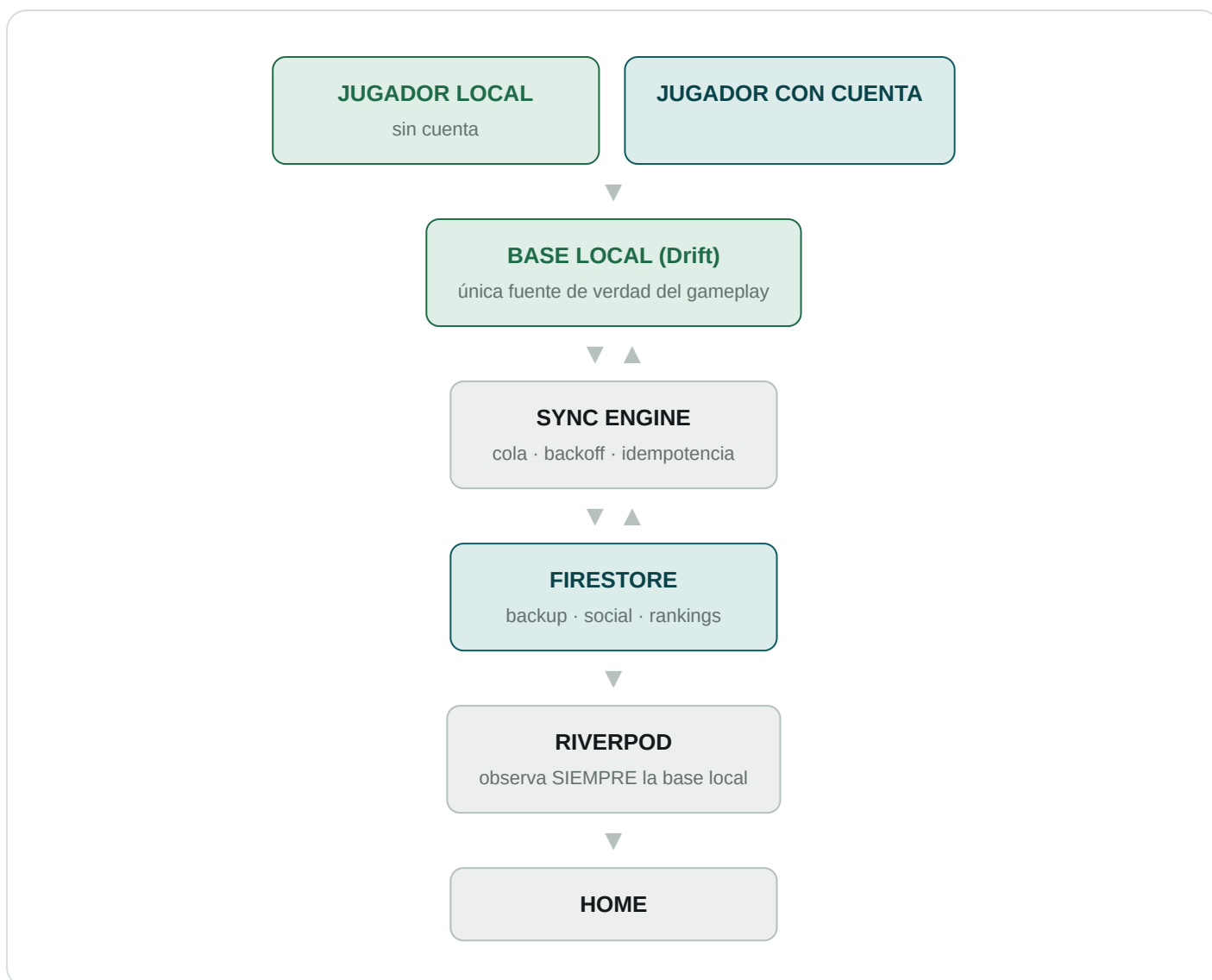
18 Propuesta de Cloud Data Model

Sin colecciones nuevas. Reasignamos responsabilidades sobre las que ya están declaradas:

RUTA	CONTIENE	VIVE EN
users/{uid}	Perfil privado y totales acumulados: totalXp, totalCoins, gamesWon, totalMoves, longestStreak, currentStreak, lastPlayedDate, email, phoneNumber	ambos
users/{uid}/matches/{matchId}	Historial. matchId = el UUID local. Inmutable.	ambos
users/{uid}/level_progress/{n}	Solo isCompleted, bestScore, completedAt	ambos
users/{uid}/sync_ops/{opId}	Ledger de idempotencia. Un doc vacío con appliedAt por operación aplicada	solo nube
users/{uid}/achievements/{id}	Logros desbloqueados (hoy se calculan en cliente y no se guardan)	ambos
user_index/{uid}	Perfil público: solo displayName, avatarSeed, level, totalXp. Es lo que sustituye al read: if true sobre el doc privado	solo nube
daily_challenges/{id}	Definición del reto. Solo lectura, se cachea localmente	nube → caché local
leaderboards/{id}	Rankings globales. Solo lectura, nunca autoridad local	solo nube
—	Definiciones de niveles (tiempo, nº de cartas)	solo local — son deterministas
—	Vidas y su temporizador	solo local en fase 1

Regla de reparto: **lo determinista se genera, lo acumulado se sincroniza con increments, lo derivado se recalcula, y lo social vive solo en la nube.**

19 Propuesta de Sync Engine



Tres piezas en `lib/core/sync/`:

- **SyncQueue** — DAO sobre `sync_operations`. Encolar, tomar el siguiente lote elegible (`status = pending AND nextAttemptAt <= now`), marcar resultado.
- **SyncEngine** — el orquestador. Se despierta cuando la conectividad pasa a `online`, cuando el usuario inicia sesión, cuando la app vuelve a primer plano y cuando se encola una operación estando en línea. Procesa **en serie**, respetando el orden de creación.
- **SyncStatusProvider** — expone `synced | pending | syncing | failed | offline` a partir del recuento de la cola y del estado de red, para que la UI lo muestre en lenguaje humano.

Escritura de una recompensa, paso a paso

```
// Todo esto ocurre en UNA transacción local. Sin red. Sin await de Firebase.
db.transaction(() {
  updatePlayer(totalXp += 120, totalCoins += 50, gamesWon += 1);
  insertMatch(match); // id ya existía desde el inicio
  enqueue(SyncOperation(
    opId: uuid(), type: 'RECORD_WIN',
    payload: {matchId, xp: 120, coins: 50, moves: 24},
  ));
});
// La UI ya se actualizó. El motor sincroniza cuando pueda.
```

Idempotencia

El patrón: **el opId es el ID del documento del ledger, y el ledger se escribe en la misma transacción que el efecto.**

```
firestore.runTransaction((tx) {
  final ledger = users/{uid}/sync_ops/{opId};
  if ((await tx.get(ledger)).exists) return; // ya aplicada → no-op
  tx.update(users/{uid}, {totalXp: increment(120), totalCoins: increment(50)});
  tx.set(users/{uid}/matches/{matchId}, match); // mismo id → sobrescribe, no duplica
  tx.set(ledger, {appliedAt: serverTimestamp(), type: 'RECORD_WIN'});
});
```

Si la respuesta se pierde y reintentamos, la transacción encuentra el ledger y no hace nada. Resuelve exactamente el escenario de tu punto 33.

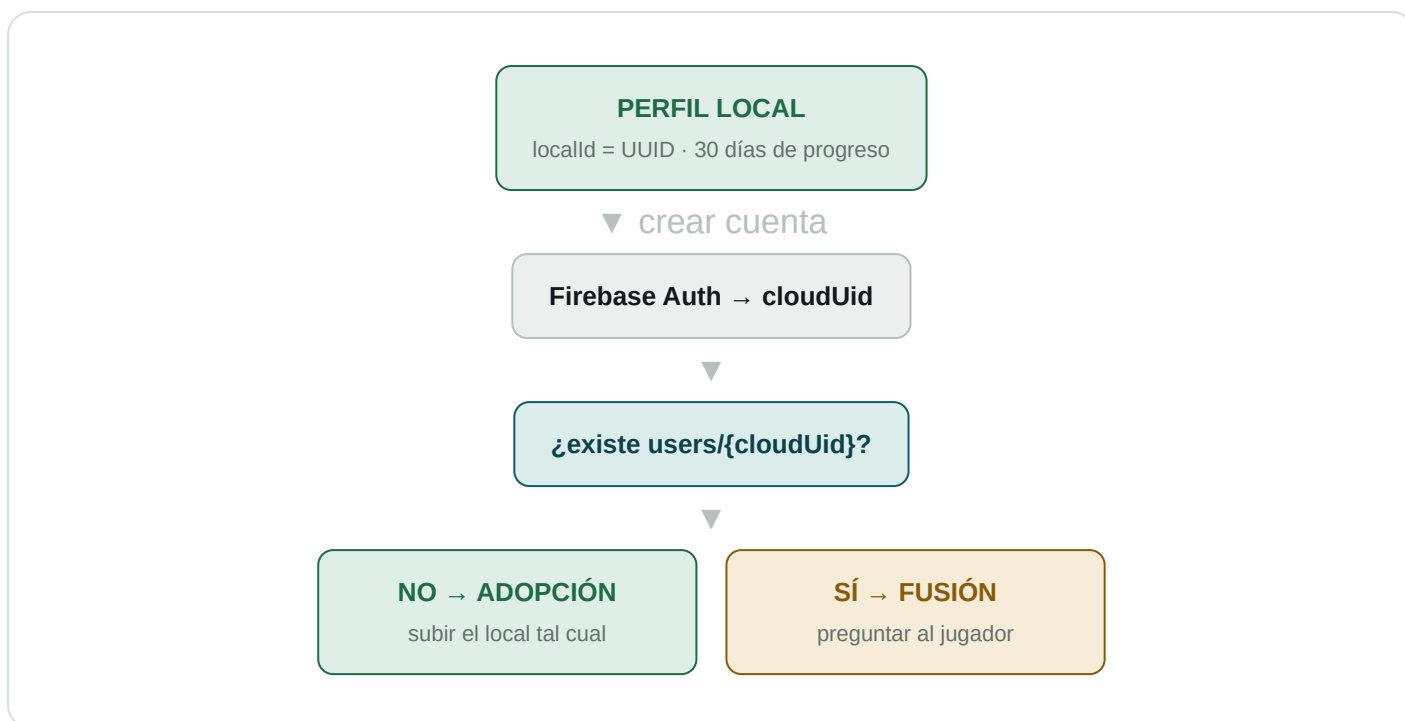
Reintentos

TIPO DE FALLO	ACCIÓN
Red / unavailable / deadline-exceeded	Backoff exponencial con jitter: 2s, 8s, 30s, 2min, 10min, 1h. Tope 8 intentos, luego failed y espera al siguiente evento de conectividad.
permission-denied / invalid-argument	No reintentar. Marcar failed con el motivo. Reintentar solo lo hace peor y quema cuota.
Sin sesión	La operación se queda pending indefinidamente. Es el caso normal del jugador local: la cola es su backlog para cuando cree cuenta.

Cómo se ve en pantalla

ESTADO	CON CUENTA	SIN CUENTA
synced	✓ Guardado	 Guardado en este dispositivo (constante; sin ruido de sincronización)
syncing	 Sincronizando...	
pending / offline	 Guardado en este dispositivo	
failed	 Guardado aquí · lo intentaremos de nuevo	

Ningún estado dice «error». Para el jugador, su progreso *siempre* está guardado — la única variable es dónde.



Identidad. El `localId` se genera en el primer arranque y no cambia nunca. Crear cuenta *añade* un `cloudUid` a la misma fila; no crea un perfil nuevo. Es lo que garantiza que nada se pierde por definición: no hay traspaso, hay vinculación.

Caso adopción (el 95 %: cuenta nueva, sin documento previo). Se marca el perfil como `migrating`, se generan operaciones de sincronización para el estado completo — totales, todas las partidas, todo el progreso de niveles — y el motor las procesa. Al confirmarse la última, se marca `synced`. **Nada local se borra jamás**; la base local sigue siendo la fuente de verdad del gameplay para siempre.

Caso fusión (el jugador entra con una cuenta que ya tiene progreso, p. ej. desde otro teléfono). Aquí no hay respuesta técnicamente correcta, y adivinar es cómo se pierde el progreso de alguien. Se pregunta, en lenguaje humano:

DIÁLOGO PROPUESTO

Ya tienes un progreso guardado

En esta cuenta: Nivel 12 · 4.300 monedas

En este dispositivo: Nivel 8 · 1.250 monedas

[Continuar con el de la cuenta] [Usar el de este dispositivo]

Rollback. No hace falta uno explícito, y esa es la ventaja del diseño: como no borramos nada y las operaciones son idempotentes, una migración interrumpida a la mitad simplemente se reanuda. Si el usuario cierra la app con 40 de 120 partidas subidas, al volver quedan 80 operaciones **pending** y el motor sigue donde estaba.

Duplicados. Imposibles por construcción: cada partida viaja con el UUID que ya tenía en local, así que subirla dos veces sobrescribe el mismo documento. Los totales viajan por el ledger de **opId**.

21 Estrategia de conflictos

Cuatro clases de dato, cuatro reglas. La mayoría de los conflictos no se resuelven: se hacen imposibles.

CLASE	EJEMPLOS	REGLA
Acumulativos	<code>totalXp</code> , <code>totalCoins</code> , <code>gamesWon</code> , <code>totalMoves</code>	Basado en operaciones. Se sincroniza el <i>delta</i> , no el total, con <code>FieldValue.increment</code> . Conmutativo: el orden da igual y ambos dispositivos convergen.
Derivados	<code>level</code> , progreso de la barra, <code>rank</code>	No se sincronizan. Se recalculan desde <code>totalXp</code> . Un dato que no viaja no puede entrar en conflicto.
Críticos de valor único	<code>displayName</code> , <code>avatarSeed</code>	Last-write-wins con <code>version</code> + <code>updatedAt</code> . Gana el <code>version</code> mayor; el empate lo rompe <code>updatedAt</code> .
Colecciones	<code>matches</code> , <code>level_progress</code>	ID único por documento. Las partidas son inmutables → sin conflicto. En <code>level_progress</code> gana el <code>bestScore</code> más alto (max, no LWW).

TU EJEMPLO, RESUELTO

Dispositivo A offline gana 450 XP; dispositivo B offline gana 620 XP. Con totales, uno pisa al otro y se pierden 450 o 620. Con deltas, A envía `increment(+450)` y B `increment(+620)`: el servidor queda en +1.070, que es lo que el jugador se ganó de verdad. El nivel se recalcula solo en ambos dispositivos al recibir el total. Nadie sobrescribe a nadie porque nadie envía un total.

La racha, que es el caso raro

Ni acumulativa ni LWW. Se guardan tres campos: `currentStreak`, `longestStreak` y `lastPlayedDate` como `'YYYY-MM-DD'` en zona local — texto, no timestamp, precisamente para que funcione sin servidor y sobreviva a cambios de huso horario. La transición al jugar:

SITUACIÓN	RESULTADO
<code>lastPlayedDate</code> es null	<code>currentStreak = 1</code>
Mismo día	sin cambios
Ayer	<code>currentStreak += 1</code>
Hace 2+ días	<code>currentStreak = 1</code>
Fecha futura (reloj manipulado)	sin cambios; se registra para revisión

Al fusionar: `longestStreak = max(local, nube)`, y `currentStreak` se toma del que tenga `lastPlayedDate` más reciente. Es una regla de dos líneas y cubre los seis casos de tu punto 15.

Lo que NO hacemos en fase 1: nada de CRDTs, vector clocks ni logs de operaciones bidireccionales. La combinación de «acumulativos por delta + derivados recalculados + colecciones por ID» cubre el 95 % de los casos reales con complejidad casi nula. El `version` por fila queda ahí para cuando haga falta algo más fino.

LO PRIMERO, QUE NO ES DE INTEGRIDAD SINO DE PRIVACIDAD

Quitar `allow read: if true` de `users/{uid}` y de `matches`. Hoy cualquiera con el ID del proyecto puede leer el email y el teléfono de todos los jugadores. Los datos públicos que amigos y rankings necesitan se mueven a `user_index/{uid}` — colección que **ya está declarada** en tus rules y que nadie usa todavía.

Fase 1 — lo que las rules sí pueden hacer

CONTROL	CÓMO
Allowlist de campos	Solo se aceptan las claves conocidas. Cualquier campo desconocido rechaza la escritura entera.
Monotonía de acumulados	<code>request.resource.data.totalXp >= resource.data.totalXp</code> . Ídem <code>gamesWon</code> , <code>totalMoves</code> , <code>longestStreak</code> . Nadie puede bajar sus totales para reordenar un ranking.
Tope por escritura	El delta de <code>totalXp</code> no puede superar un máximo plausible por operación. Convierte «me pongo 10 millones de XP» en «necesito 10.000 escrituras detectables».
Campos inmutables	<code>createdAt</code> y <code>email</code> no se pueden modificar tras la creación.
<code>user_index</code> derivado	Escritura solo del propio <code>uid</code> , y únicamente los cuatro campos públicos.
Partidas inmutables	<code>allow create</code> sí; <code>allow update</code> , <code>delete</code> no. Una partida escrita no se reescribe.
<code>sync_ops</code>	Crear sí, actualizar y borrar no. Si se pudieran borrar, se podría replicar una operación y duplicar recompensas.
<code>levels</code>	Añadir el bloque que falta hoy en las rules.

SÉ HONESTO SOBRE EL LÍMITE

Ninguna regla de Firestore puede hacer confiable un saldo que calcula el cliente. Las reglas solo ven la escritura, no la partida que la justifica. Con reglas puedes *acotar el daño* — monotonía, topes, allowlist — pero un cliente modificado siempre podrá inventarse una partida plausible. La única solución real es que el servidor recalcule, y eso son Cloud Functions.

Fase 2 — la evolución, ya preparada

El diseño deja el camino abierto sin trabajo extra: el cliente escribiría **solo** en `users/{uid}/sync_ops/{opId}` — que es la partida jugada, no el premio — y una Cloud Function con `onCreate` validaría la coherencia (score alcanzable con esos movimientos, tiempo transcurrido creíble, ritmo de operaciones razonable) y sería la única que escribe `totalXp` y `totalCoins`. Ese día, `users/{uid}` pasa a `write: if false` para el cliente y el problema desaparece.

Es una migración de reglas y una función, sin tocar el cliente, **porque el ledger de operaciones ya existe desde la fase 1**. Ese es el motivo de diseñarlo ahora aunque hoy solo sirva para idempotencia.

OPERACIÓN	CONFIABLE EN CLIENTE (FASE 1)
Progreso de niveles, bestScore, historial	Sí — bajo valor, autocontenido
Racha	Aceptable — manipulable con el reloj, impacto bajo
XP y nivel	Aceptable con topes — afecta a rankings
Monedas ganadas	No, si algún día hay compras reales
Monedas gastadas / compras	No — requiere validación de servidor
Resultados de multijugador	Nunca — arbitraje de servidor obligatorio

Ejecución

23 Plan de implementación por commits

Diecisiete commits en cinco bloques. Cada uno compila, pasa `analyze` y deja la app jugable. Ningún commit rompe funcionalidad existente.

C01	F4	<code>chore(db)</code>: añade Drift y el esqueleto de <code>core/database</code> Dependencias, <code>AppDatabase</code> vacía con <code>schemaVersion: 1</code>, apertura en <code>main</code>, un test que abre una base en memoria. Nada la usa todavía.
C02	F4	<code>feat(db)</code>: define las tablas del modelo local Las ocho tablas de §17 con sus índices. Solo esquema y DAOs generados.
C03	F7	<code>feat(player)</code>: perfil local con identidad propia <code>PlayerRepository</code>, generación del <code>localId</code> en el primer arranque, <code>localPlayerProvider</code>. Todavía nadie lo consume.
C04	F7	<code>refactor(model)</code>: <code>totalXp</code> monótono, nivel derivado Cambio del modelo de §17 más <code>levelFromTotalXp()</code> en un único sitio. Incluye la migración de los usuarios existentes y sus tests. Arregla el bug del nivel que nunca sube.
C05	F8	<code>feat(startup)</code>: arranque sin bloqueo y sin login obligatorio <code>main.dart</code> abre la base y llama a <code>runApp</code>; Firebase se inicializa en segundo plano. El splash enruta a Home si hay perfil local. Aquí la app se vuelve jugable sin cuenta por primera vez.
C06	F8	<code>refactor(home)</code>: la Home lee de la base local <code>homeSummaryProvider</code> y <code>homeControllerProvider</code> pasan a observar Drift. Cero cambios en widgets.
C07	F7	<code>fix(wallet)</code>: el saldo persiste de verdad Reescritura sobre el repository local. <code>spend()</code> descuenta y guarda. Arregla que gastar monedas no hiciera nada.

C08	F7	<code>fix(game)</code>: una única fórmula de recompensa Se elimina el cálculo de <code>BoardController</code> y el abono fijo de 50 monedas de <code>BoardPage.Match.calculateRewards</code> queda como fuente única. Arregla que perder por tiempo diera monedas.
C09	F7	<code>feat(game)</code>: las partidas se guardan localmente El <code>matchId</code> se genera al iniciar la partida. Escritura local transaccional. Se retira el <code>try/catch + print</code>.
C10	F7	<code>feat(streak)</code>: racha diaria funcional y offline Las cinco transiciones de §21. Implementa una funcionalidad que hasta ahora solo existía como campo vacío.
C11	F7	<code>fix(levels)</code>: definiciones locales, progreso persistido Se retiran las 50 escrituras de registro. Se corrige <code>bestScore</code> (int, con <code>max</code>) y el <code>batch.update</code> sobre documentos inexistentes. <code>LevelMapController</code> se conecta al sistema real.
C12	F7	<code>fix(lives)</code>: vidas persistentes con recarga por reloj Se guarda <code>lastRefillAt</code> y se recalcula al arrancar, en vez de reiniciar a 5.
C13	F9	<code>feat(sync)</code>: cola de operaciones <code>SyncOperation</code>, <code>SyncQueue</code> y encolado desde todas las escrituras locales. Todavía no envía nada — solo se llena y se puede inspeccionar en los tests.
C14	F10	<code>feat(sync)</code>: motor con idempotencia y backoff <code>SyncEngine</code> completo con el ledger <code>sync_ops</code>, disparadores de conectividad, sesión y ciclo de vida, y la política de reintentos de §19.
C15	F10	<code>feat(ui)</code>: estado de guardado en lenguaje humano <code>SyncStatusProvider</code> y su indicador discreto en la Home, construido con <code>AppBadge</code>. Multijugador comunica que necesita conexión sin bloquear nada más.
C16	F11	<code>feat(account)</code>: vinculación de perfil local con cuenta Adopción y fusión de §20, con el diálogo de elección. La propuesta de valor «Guarda tu progreso» aparece por primera vez, con «Ahora no».

C17 F12

`fix(security)`: cierra la fuga y valida las escrituras

Reglas de §22, `user_index` como perfil público, bloque de `levels`, y

`OFFLINE_FIRST.md` más las actualizaciones de `FIRESTORE_SETUP.md`.

Los tests (fase 13) no van en un commit propio: cada uno de C02 a C16 incluye los suyos. Es la única manera de que la suite sirva de algo, y con Drift en memoria y los repositories ya inyectables, ninguno necesita Firebase real.

EL PUNTO DE INFLEXIÓN

Merece la pena verlo: en C05 la app ya arranca sin red y sin cuenta. En C12 el bucle completo de juego —jugar, ganar XP, subir de nivel, ganar y gastar monedas, mantener la racha, ver el historial— funciona íntegramente offline, y todavía no hemos escrito una sola línea de sincronización. Firebase entra en C13 como lo que debe ser: *una mejora*, no un requisito.

Auditoría técnica de Memory Companion · rama develop · el plan comienza en FASE 1 cuando lo apruebes.